

RAG-Augmented Multi-Agent LLM Framework for Explainable Software Vulnerability Detection Using CWE Knowledge Bases

Richa Verma (25MCSS02) Advisor: Dr. Akshay Pandey

Hard examples and a live model

Where this fits

The pipeline can already mark lines, search the CWE book, fill the answer form, and check that form. On our first 12 files the simple checker looked perfect, because those files were easy.

This note is about 24 harder files we wrote on purpose, and about a live language model. These files are still ours. They are not a public test set.

What we did

Twelve of the new files are unsafe but quiet. Twelve are safe but look risky, often because a comment or a name mentions a flaw. A pattern checker that hunts for those words will get both kinds wrong.

On the original text, the simple checker scores $F1 = 0$. It raises 12 false alarms and misses 12 bugs. F1 is the combined score of false alarms and missed bugs. Zero means it failed both ways.

Before the model sees a file, we blank comments and rename obvious labels such as bad and good. That stops the question from containing the answer. After that cleaning, the template still scores $F1 = 0$. It has fewer false alarms, because comment-only hits disappear, but it still misses every bug.

The live model is Groq, model openai/gpt-oss-20b. It writes the same answer form. On these 24 files its F1 is 0.923: 2 false alarms and no missed bugs.

We ran the same 24 again with CWE search turned off. The F1 stayed 0.923. So, on this particular set, showing the model the book pages did not change detection. We do not treat that as proof that search is useless in general. We also do not treat it as proof that search helps.

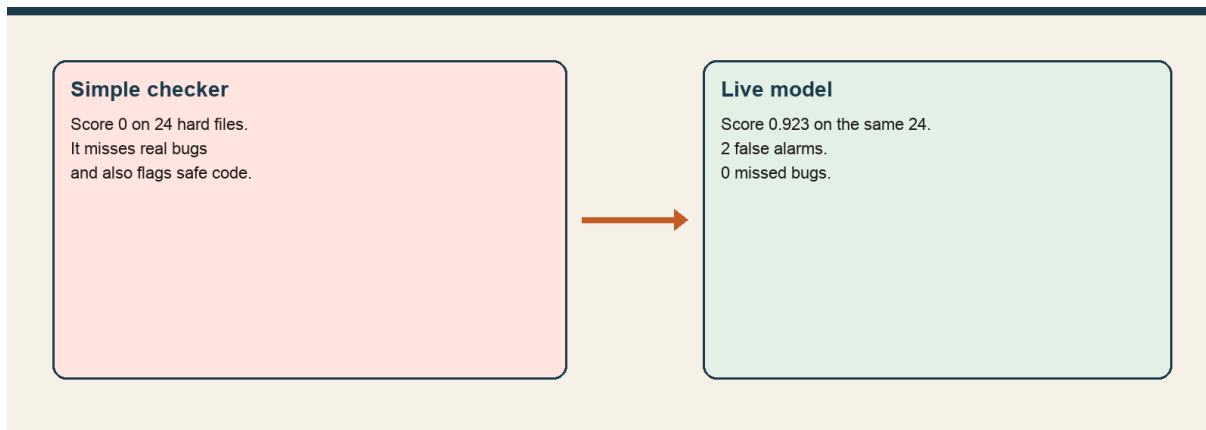
A concrete example

One safe file has a comment that says the code is flawed, but the code itself is fine. The simple checker reads the comment and raises a false alarm. After we blank comments, that alarm can disappear. The template still does not understand a quiet bug.

One unsafe file never uses the obvious words. The pattern checker misses it. The live model reads the method and can still call it unsafe and name a CWE.

Turning the book search off did not change the 0.923 score. The model was already getting the decision right from the code. The book may still matter for the explanation, but this test does not prove that.

Picture



The simple checker fails these traps. The live model recovers most of them.

The numbers

System	F1	False alarms	Missed bugs	Files
Simple checker, original text	0.000	12	12	24
Template, after cleaning	0.000	6	12	24
Live model	0.923	2	0	24
Live model, search off	0.923	2	0	24

The live model caught every unsafe file in this set and was wrong on 2 safe files. Precision is 0.857 and recall is 1.000. Those two numbers combine into F1 0.923.

Closing

This is a useful contrast: rules fail, the model mostly recovers. It is not a public result.

The next measurement should be the simple checker on named public Java sets, reported separately from these 24 files.

What we are not saying

These 24 files are not Juliet. We do not say that CWE search improves detection.